

M13 Review / Repair - Teaching RC2

OPEN ONLY AFTER SAVED ATTEMPT

P01 repair - Data Lake vs Warehouse vs Lakehouse

You already know Parquet; M13 adds reliable table operations/version history over file storage.

Repair principle: Data lake is file/object-storage-oriented and flexible.

Key proof: catalog/governance -> discovery/access management

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P02 repair - Why plain files are not tables

Plain files have no single atomic table commit record; readers can list different file sets while a write is in progress.

Repair principle: A file write/rename may be atomic at object level but a table update can involve many files.

Key proof: history -> version log

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P03 repair - Transaction log, ACID and table versions

ACID terms are useful only if you can explain what they mean for a failed/concurrent file-table write.

Repair principle: Atomicity: transaction changes become visible as committed version together or not.

Key proof: D -> committed log/data persist

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P04 repair - Create, read, update and delete

File-based update/delete rewrites underlying data files. Delta provides table operations, but you still need correct predicates and controls.

Repair principle: Create/write modes must be explicit (error/append/overwrite) and safe for target path.

Key proof: history -> new version per commit

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P05 repair - Schema enforcement and schema evolution

Delta can reject incompatible writes, preventing accidental table corruption; evolution convenience must remain intentional.

Repair principle: Schema enforcement checks writes against current table schema.

Key proof: downstream -> compatibility re-tested

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P06 repair - Time travel and version history

When a bad update lands, time travel lets you inspect before/after state without restoring an entire backup first.

Repair principle: Delta history lists version, timestamp, operation and operation metrics depending runtime.

Key proof: retention dependency -> old files must still exist

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P07 repair - MERGE, upserts and idempotent incremental loads

Incremental batches often repeat keys/replay. A MERGE can make current-state application safe, but only if match and version logic are correct.

Repair principle: Match on stable business key.

Key proof: exact replay -> stable state

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P08 repair - Slowly Changing Dimension Type 2 with Delta

Current-state MERGE loses attribute history; SCD2 keeps historical dimension versions for time-aware analytics.

Repair principle: SCD2 row has business key, attributes, effective_from/effective_to and is_current (often surrogate key).

Key proof: replay -> no extra duplicate version

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.

P09 repair - Bronze, Silver and Gold layer contracts

Medallion architecture is only useful if layers have meaning, not if every pipeline copies the same data three times.

Repair principle: Bronze preserves source-like data + ingestion metadata and supports replay.

Key proof: Gold -> consumer-ready modeled/aggregated

STOP - check your understanding

Close Review and solve a changed case.

Answer in your own words before continuing.